

PLANEJAMENTO DE DESENVOLVIMENTO

Modelos, Métodos e Técnicas de Engenharia de Software
Melhoria do Ticket SIR — Sistema de Intervenção de Rede
UniBH — 2026

Este documento descreve as atividades planejadas para o desenvolvimento do trabalho prático, organizadas por área. O foco da implementação é a aplicação de princípios SOLID, padrões de projeto e testes unitários.

1. AJUSTES NO MODELO DE DADOS

1. Adicionar os campos ausentes na entidade SeguimentoTicket:
 - a. condição do equipamento (em serviço / fora de serviço);
 - b. tipo de atividade (obra, remanejamento, adequação, manutenção corretiva, manutenção preventiva);
 - c. descrição da atividade;
 - d. classificação (com interrupção / sem interrupção);
 - e. datas de início e término da execução;
 - f. datas de início e término da interrupção.
2. Criar a entidade Circuito com os campos: designação, descrição, e-mail de contato e telefone.
3. Criar a associação entre SeguimentoTicket e Circuito (um ticket pode ter múltiplos circuitos).
4. Criar os enumeradores: CondicaoEquipamento, TipoAtividade, ClassificacaoAtividade e TipoNotificacao.

2. PADRÕES DE PROJETO

1. **Observer** (comportamental): ao alterar o status de um ticket, publicar um evento de aplicação. Listeners separados tratam o envio de e-mail e o registro no histórico, sem que o serviço principal conheça os detalhes de cada ação.
2. **Strategy** (comportamental): definir uma interface de notificação com uma implementação via e-mail. A interface foi pensada de forma que, futuramente, outros canais possam ser incorporados sem necessidade de alterar o código já existente.

3. **Builder** (criacional): utilizar o padrão para construir as mensagens de notificação, evitando construtores com muitos parâmetros.
4. **Facade** (estrutural): criar uma fachada que concentra as operações principais do sistema, simplificando o acesso pelos controllers e reduzindo o acoplamento entre camadas.

3. IMPLEMENTAÇÃO DAS CAMADAS

1. Implementar `TicketRepository` e `CircuitoRepository` com as consultas necessárias.
2. Implementar `TicketService` com as regras de negócio: criação, atualização, validação de status e verificação de prazo.
3. Implementar `CircuitoService` com suporte à associação individual e à importação em massa (bulk import via texto ou CSV).
4. Implementar `EmailService` com agrupamento de circuitos por cliente, garantindo um único e-mail por atividade por destinatário.
5. Implementar `NotificacaoScheduler` para detecção de extrapolação de prazo e alerta antecipado de atividades.
6. Implementar os endpoints nos controllers de ticket e circuito.
7. Criar os DTOs de entrada e saída da API.
8. Configurar a segurança da aplicação (autenticação básica).
9. Implementar tratamento global de erros.
10. Preencher o template de e-mail com os dados da atividade e dos circuitos afetados.

4. TESTES UNITÁRIOS

1. Testes para `TicketService`: criação de ticket, transições de status válidas e inválidas, e validação de prazo.
2. Testes para `EmailService`: agrupamento correto de circuitos por destinatário.
3. Testes para `CircuitoService`: associação de circuitos e importação em massa com validação de duplicatas.
4. Testes para os listeners do Observer: verificar se o envio de e-mail e o registro de histórico são acionados corretamente na mudança de status.

5. DOCUMENTAÇÃO (POR FAZER)

1. Formalizar as User Stories com critérios de aceitação para as funcionalidades implementadas.
2. Atualizar o diagrama de classes do sistema.
3. Elaborar o diagrama de casos de uso.
4. Atualizar o diagrama de sequência para o fluxo de notificação.
5. Escrever o README.md com instruções de configuração e execução da aplicação.
6. Documentar a justificativa dos padrões SOLID e design patterns aplicados.